

Chapter 15: Multiobjective Optimization

Algorithms for Optimization, 2nd ed. (Kochenderfer & Wheeler, 2026)

Lecture Notes

2026

Outline

- 1 Introduction
- 2 Pareto Optimality
- 3 Constraint Methods
- 4 Weight Methods
- 5 Multiobjective Population Methods
- 6 Preference Elicitation
- 7 Summary & Exercises

Motivation: Why Multiple Objectives?

- Single-objective optimization: one scalar $f(x) \in \mathbb{R}$.
- Reality: engineering involves **competing objectives**.
 - Cost vs. performance
 - Speed vs. accuracy
 - Alert rate vs. collision rate

- **Multiobjective** (vector) optimization:

$$\min_{x \in \mathcal{X}} f(x) = [f_1(x), \dots, f_m(x)]^\top$$

- Two broad strategies:
 - 1 **Scalarize** f into a single objective.
 - 2 Find the **Pareto frontier** — the set of all best trade-offs.

Criterion Space $\mathcal{Y}$

The image of the design space $\mathcal{X}$ through f :

$$\mathcal{Y} = \{f(x) : x \in \mathcal{X}\} \subseteq \mathbb{R}^m$$

Each point $y \in \mathcal{Y}$ represents one achievable trade-off.

15.1.1 Dominance

Definition: Dominance (Eq. 15.1)

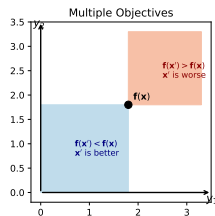
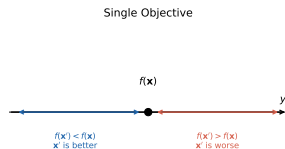
x **dominates** x' if and only if

$$f_i(x) \leq f_i(x') \quad \forall i \in 1:m$$

and $f_i(x) < f_i(x')$ for some i

Compact check: $\text{all}(y \leq y')$ and $\text{any}(y < y')$

- Single objective: always comparable.
- Multiple objectives: *ambiguity region* exists (neither dominates).



Dominance in Python

```
import numpy as np

def dominates(y: np.ndarray, y_prime: np.ndarray) -> bool:
    """Return True if y dominates y_prime (minimization)."""
    return bool(np.all(y <= y_prime) and np.any(y < y_prime))

# Example: 2 objectives
y = np.array([1.0, 3.0])
y1 = np.array([2.0, 4.0]) # dominated by y
y2 = np.array([0.5, 5.0]) # not dominated by y (y2[0] < y[0])
y3 = np.array([0.5, 2.0]) # dominates y

print(dominates(y, y1)) # True -- y is better in both
print(dominates(y, y2)) # False -- y2 wins on f1
print(dominates(y3, y)) # True -- y3 dominates y
```

Key takeaway

x dominates $x' \Rightarrow x$ is strictly better.

No dominance in either direction $\Rightarrow$ ambiguity region.

15.1.2 Pareto Frontier

Pareto Optimality

$x \in \mathcal{X}$ is **Pareto-optimal** if no $x' \in \mathcal{X}$ dominates it.

Pareto Frontier

The set of all Pareto-optimal points. Every point on the frontier lies on the boundary of $\mathcal{Y}$.

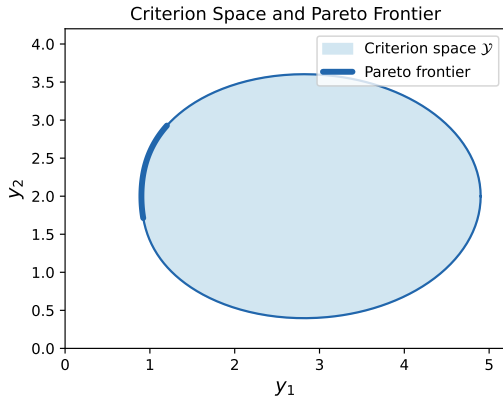
Weakly Pareto Optimal

x is **weakly Pareto-optimal** if $\nexists x'$ such that $f(x') < f(x)$ (all objectives strictly better).

Pareto-optimal $\Rightarrow$ weakly Pareto-optimal, but not vice versa.

Utopia Point (Eq. 15.2)

$$y_i^{\text{utopia}} = \min f_i(x)$$



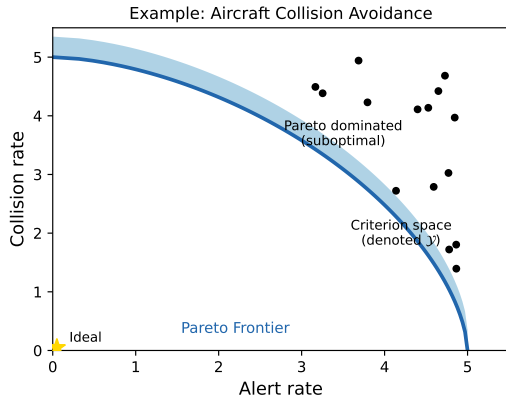
Example 15.1: Aircraft Collision Avoidance

- Two competing objectives:
 - 1 **Minimize** collision rate f_1 .
 - 2 **Minimize** alert rate f_2 .
- More alerts $\rightarrow$ fewer collisions *if* pilots comply.
Too many alerts $\rightarrow$ alert fatigue, reduced compliance.
- Designers must carefully trade these off.
- The Pareto frontier reveals all achievable trade-offs.

What the Pareto frontier tells us

Any design *inside* the frontier (dominated) is strictly improvable.

Only the frontier points are of interest.



Naive Pareto Filter (Algorithm 15.2)

```
import numpy as np

def dominates(y, y_prime):
    return np.all(y <= y_prime) and np.any(y <
        y_prime)

def naive_pareto(xs, ys):
    """
    xs: list of design points
    ys: list of objective vectors
    Returns (pareto_xs, pareto_ys) -- non-dominated
        points.
    """
    pareto_xs, pareto_ys = [], []
    for x, y in zip(xs, ys):
        # keep if no other y' dominates y
        if not any(dominates(yp, y) for yp in ys):
            pareto_xs.append(x)
            pareto_ys.append(y)
    return pareto_xs, pareto_ys

# Example
xs = np.random.rand(200, 2)
ys = [np.array([np.linalg.norm(x),
                np.linalg.norm(x - 1)]) for x in xs]
px, py = naive_pareto(xs, ys)
print(f"Pareto points: {len(py)} of {len(ys)}")
```

Complexity

- n design points, m objectives.
- Naive check: $O(n^2m)$.
- Wasteful: needs many samples to approximate frontier well.
- Does not guarantee smooth or complete frontier.

Limitation

Random sampling is inefficient — most points will be dominated. Better methods use structured search.

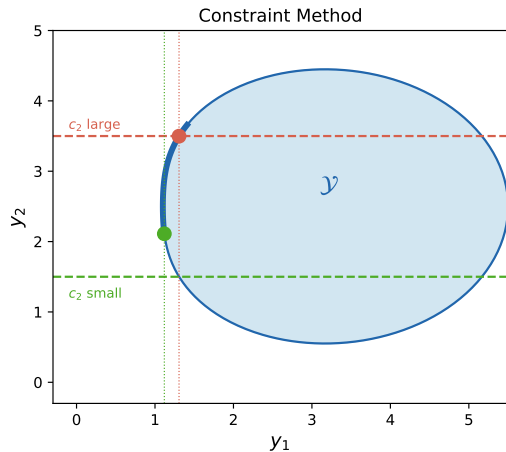
15.2.1 Constraint Method (ε -Constraint)

Formulation (Eq. 15.3)

Minimize f_1 subject to upper bounds on all other objectives:

$$\begin{aligned} \min_x \quad & f_1(x) \\ \text{s.t.} \quad & f_2(x) \leq c_2 \\ & \vdots \\ & f_m(x) \leq c_m \\ & x \in \mathcal{X} \end{aligned}$$

- Varying c traces the Pareto frontier.
- **Advantage:** can find Pareto points in *nonconvex* regions.
- Each value of c gives one solve of a standard optimization.



As c_2 decreases from large to small, the optimal point traces the Pareto frontier.

Constraint Method in Python

```
import numpy as np
from scipy.optimize import minimize

def constraint_method_2d(f1, f2, c2_vals, x0=0.0):
    """
    Sweep c2 values to trace the Pareto frontier (2 objectives).
    f1, f2: scalar functions of x
    c2_vals: array of constraint bounds on f2
    """
    pareto_pts = []
    for c2 in c2_vals:
        result = minimize(
            f1, x0,
            constraints={'type': 'ineq', 'fun': lambda x: c2 - f2(x)},
            method='SLSQP'
        )
        if result.success:
            x_opt = result.x[0]
            pareto_pts.append((f1(x_opt), f2(x_opt)))
    return np.array(pareto_pts)

# Exercise 15.7: minimize  $[x^2, (x-2)^2]$ 
f1 = lambda x: x**2
f2 = lambda x: (x - 2)**2
c2_vals = np.linspace(0.01, 4.0, 50)
front = constraint_method_2d(f1, f2, c2_vals, x0=np.array([1.0]))
```

15.2.2 Lexicographic Method

Procedure

Rank objectives in order of importance. Solve sequentially:

- ➊ $\min_x f_1(x), x \in \mathcal{X} \Rightarrow y_1^*$
- ➋ $\min_x f_2(x) \text{ s.t. } f_1(x) \leq y_1^*, x \in \mathcal{X} \Rightarrow y_2^*$
- ➌ $\min_x f_3(x) \text{ s.t. } f_1 \leq y_1^*, f_2 \leq y_2^*, x \in \mathcal{X} \Rightarrow y_3^*$
- ➍ $\vdots$

Each step preserves the previous optimal.

- Always feasible: previous minimum is always feasible.
- Produces a **single** Pareto-optimal point.
- Sensitive to objective ordering.
- Inequalities are preferred (robustness to suboptimal solvers).

Limitation

Produces only *one* design, not the full frontier. Different orderings give different results.

Use case

When there is a clear priority ranking among objectives (e.g., safety first, then efficiency, then cost).

15.3.1 Weighted Sum Method

Weighted Sum Objective (Eq. 15.4)

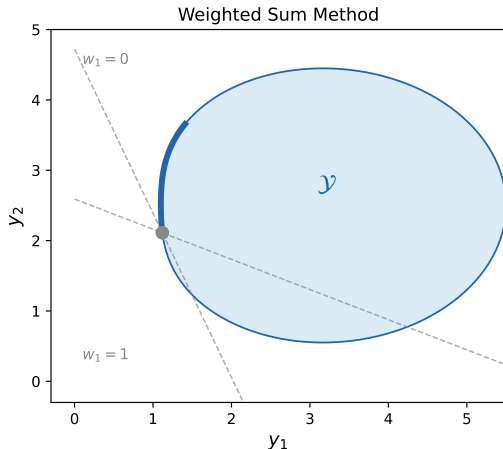
$$f(x) = w^\top f(x) = \sum_{i=1}^m w_i f_i(x)$$

where $w \geq 0$, $\sum_i w_i = 1$.

- Each w_i = cost associated with objective i .
- Sweep w over the simplex to trace the frontier.
- In 2D: vary $w_1 \in [0, 1]$, $w_2 = 1 - w_1$.

Critical Limitation

Cannot recover Pareto points in **nonconvex** regions of the frontier. Linear iso-contours miss concave sections.



Weighted Sum in Python

```
import numpy as np
from scipy.optimize import minimize

def weight_pareto(f1, f2, n_pts=50, x0=0.0):
    """
    Weighted sum method: sweep w1 from 0 to 1.
    Returns list of (y1, y2) Pareto-optimal points.
    """
    results = []
    for w1 in np.linspace(0, 1, n_pts):
        w2 = 1.0 - w1
        obj = lambda x: w1 * f1(x) + w2 * f2(x)
        res = minimize(obj, x0, method='BFGS')
        if res.success:
            x_opt = res.x[0]
            results.append((f1(x_opt), f2(x_opt)))
    return np.array(results)

# Example: f1(x) = x^2, f2(x) = (x-2)^2
f1 = lambda x: float(x**2)
f2 = lambda x: float((x - 2)**2)
front = weight_pareto(f1, f2, n_pts=30, x0=np.array([1.0]))
# Note: misses nonconvex portions!
```

Numerical example (3 points)

15.3.2 Goal Programming

L_p Goal Programming (Eq. 15.5)

Minimize the L_p distance to a goal point y^{goal} :

$$\min_{x \in \mathcal{X}} \|f(x) - y^{\text{goal}}\|_p$$

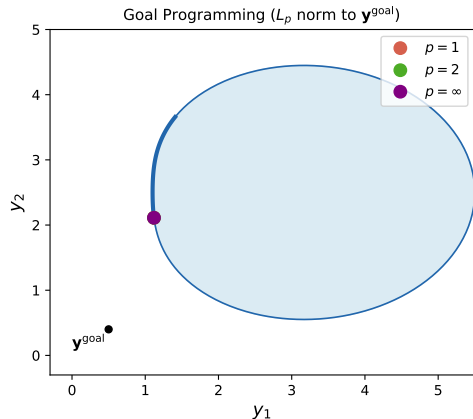
Goal point is typically the **utopia point**.

Weighted Exponential Sum (Eq. 15.6)

$$f(x) = \sum_{i=1}^m w_i |f_i(x) - y_i^{\text{goal}}|^p$$

$w \geq 0$, $\sum w_i = 1$, $p \geq 1$.

- $p = 1$: Manhattan distance (L1)
- $p = 2$: Euclidean distance
- $p \rightarrow \infty$: Chebyshev (min-max)



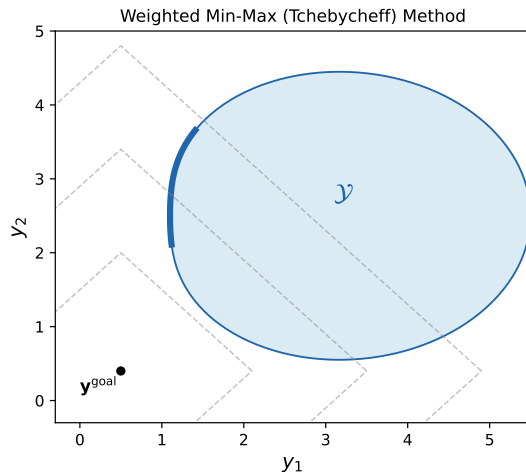
Different p values reach different Pareto-optimal points.

15.3.3–15.3.5 Advanced Weight Methods

Weighted Min-Max / Tchebycheff (Eq. 15.7)

$$f(x) = \max_i \left[w_i \left(f_i(x) - y_i^{\text{goal}} \right) \right]$$

- Limit of weighted exponential sum as $p \rightarrow \infty$.
- Contours are *diamonds* — enter nonconvex regions.
- Can find **complete** Pareto-optimal set by varying w .
- May also produce *weakly* Pareto-optimal points.



Augmented Tchebycheff (Eq. 15.8)

Exponential Weighted Criterion (Eq. 15.9)

Weighted Min-Max in Python

```
import numpy as np
from scipy.optimize import minimize

def weighted_minmax(f_list, w, y_goal, x0, rho=1e-3):
    """
    Augmented Tchebycheff (weighted min-max + small perturbation).
    f_list: list of objective functions [f1, f2, ...]
    w      : weight vector (sums to 1, all positive)
    y_goal: goal/utopia point
    rho    : small scalar (0.0001 to 0.01)
    """
    def objective(x):
        ys = np.array([fi(x) for fi in f_list])
        cheby = np.max(w * (ys - y_goal))
        return cheby + rho * np.dot(ys, y_goal)

    res = minimize(objective, x0, method='NELDER-MEAD')
    return res

# Example: 2 objectives on R^2
f1 = lambda x: x[0]**2 + x[1]**2
f2 = lambda x: (x[0]-2)**2 + (x[1]-1)**2
w = np.array([0.5, 0.5])
yg = np.array([0.0, 0.0]) # utopia point (approx)
res = weighted_minmax([f1, f2], w, yg, x0=np.array([1.0, 1.0]))
print("Optimal x:", res.x)
print("Objectives:", f1(res.x), f2(res.x))
```


Comparison of Weight Methods

Method	Convex frontier?	Nonconvex frontier?	Full coverage	Complexity
Weighted Sum	✓	×	Partial	$O(n_w \cdot T_{\text{opt}})$
Goal Programming	✓	Partial	No	$O(n_p \cdot T_{\text{opt}})$
Wtd. Exp. Sum	✓	✓	Partial	$O(n_w \cdot T_{\text{opt}})$
Wtd. Min-Max	✓	✓	Yes*	$O(n_w \cdot T_{\text{opt}})$
Constraint Method	✓	✓	Yes	$O(n_c \cdot T_{\text{opt}})$

*Augmented form avoids weakly Pareto-optimal points.

T_{opt} = cost of single optimization; n_w, n_c = number of weight/constraint values swept.

15.4 Multiobjective Population Methods

Key Idea

Population methods naturally explore multiple regions of the design space simultaneously. They can be extended to **cover the entire Pareto frontier** in a single run.

Three Main Approaches

- 1 **Subpopulations** (§15.4.1): partition population to optimize different objectives separately.
- 2 **Nondomination ranking** (§15.4.2): rank individuals by their Pareto level (NSGA-II).
- 3 **Pareto filters** (§15.4.3): maintain an elite archive of Pareto-optimal points.
- 4 **Niche techniques** (§15.4.4): encourage spread across the frontier.

Why population methods excel

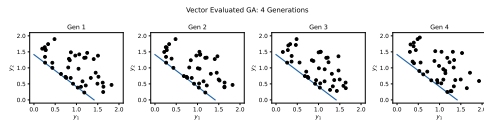
- Single run → many Pareto points.
- No scalarization needed.
- Handle nonconvex, discontinuous frontiers.
- Naturally diverse.

15.4.1 Vector Evaluated Genetic Algorithm (Algorithm 15.4)

Algorithm: VEGA

- 1 Evaluate population: $y_i = f(x_i)$ for all i .
- 2 Split into m **subpopulations** of equal size.
- 3 Each subpopulation k selects parents by fitness on f_k only.
- 4 Shuffle all parent pairs.
- 5 Apply crossover and mutation to form next generation.

- Each subpopulation specializes in one objective.
- Crossover between subpopulations maintains diversity.
- Tends to converge to extreme points of the Pareto frontier.



Population evolves toward Pareto frontier over generations.

VEGA in Python

```
import numpy as np

def tournament_select(fitness, n_select):
    """Simple tournament selection."""
    idx = np.random.choice(len(fitness), size=(n_select, 2))
    winners = idx[range(n_select),
                    np.argmin(fitness[idx], axis=1)]
    return winners

def vega_step(population, f_list):
    """One generation of VEGA."""
    m = len(f_list)
    n_pop = len(population)
    n_sub = n_pop // m
    # Evaluate all
    ys = np.array([[fi(x) for fi in f_list] for x in population])
    # Select subpopulation parents
    parents = []
    for k in range(m):
        fitness_k = ys[:, k]
        sel = tournament_select(fitness_k, n_sub)
        parents.extend(sel.tolist())
    parents = np.random.permutation(parents)
    # Crossover + mutation
    new_pop = []
    for i in range(0, n_pop, 2):
        p1, p2 = population[parents[i]], population[parents[i+1]]
```

15.4.2 Nondomination Ranking (Algorithm 15.5)

NSGA-II Ranking Levels

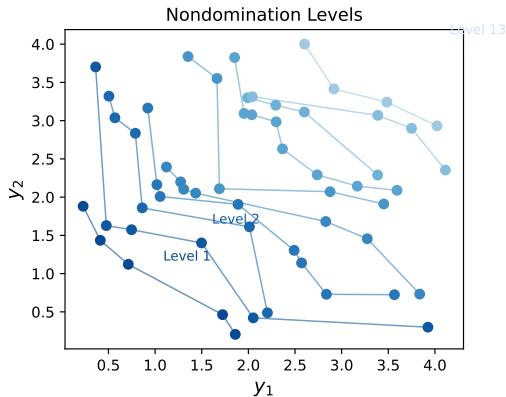
- Level 1:** Non-dominated individuals in entire population.
- Level 2:** Non-dominated after removing Level 1.
- Level k :** Non-dominated after removing Levels $1, \dots, k - 1$.

Continue until all individuals are ranked.

Selection Rule

Lower level = better. Level 1 is preferred over Level 2, etc. Within the same level, use crowding distance.

- Reference: Deb et al., IEEE Trans. Evol. Comp., 2002 (NSGA-II).



Darker = lower (better) level.

Nondomination Ranking in Python

```
import numpy as np

def dominates(y, y_prime):
    return np.all(y <= y_prime) and np.any(y < y_prime)

def nondomination_levels(ys):
    """
    Assign nondomination levels to objective vectors.
    Level 1 = best (non-dominated in full set).
    ys: (n, m) array of objective values.
    Returns: levels array of shape (n,).
    """
    n = len(ys)
    levels = np.zeros(n, dtype=int)
    L = 0
    while np.any(levels == 0):
        L += 1
        for i in range(n):
            if levels[i] == 0:
                dominated = False
                for j in range(n):
                    if j != i and (levels[j] == 0 or levels[j] == L):
                        if dominates(ys[j], ys[i]):
                            dominated = True
                            break
                if not dominated:
                    levels[i] = L
```

15.4.3 Pareto Filters (Algorithm 15.7)

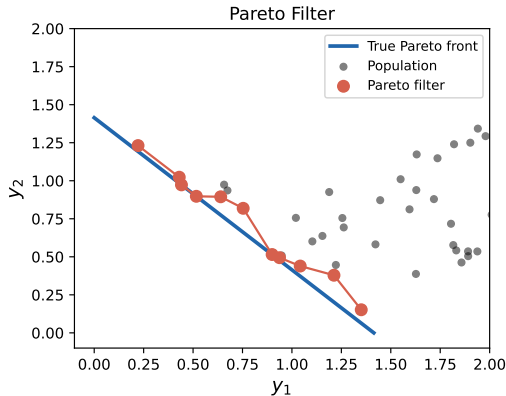
Pareto Filter: Elite Archive

- 1 For each new individual (x, y) :
 - If not dominated by any filter point, **add** it.
- 2 Remove all filter points dominated by new additions.
- 3 If filter exceeds capacity C :
 - Find the closest pair in criterion space.
 - Remove one (Algorithm 15.6).

Closest-Pair Pruning (Alg. 15.6)

$$(i^*, j^*) = \arg \min_{i < j} \|y_i - y_j\|$$

Remove one of i^*, j^* randomly.



Red dots: Pareto filter approximating the true frontier (blue).

Pareto Filter in Python

```
import numpy as np

def discard_closest_pair(xs, ys):
    """Remove one point from the closest pair in criterion space."""
    min_dist, idx = np.inf, 0
    for i in range(len(ys)):
        for j in range(i+1, len(ys)):
            d = np.linalg.norm(ys[i] - ys[j])
            if d < min_dist:
                min_dist = d
                idx = np.random.choice([i, j])
    xs.pop(idx); ys.pop(idx)

def update_pareto_filter(filter_xs, filter_ys, xs, ys,
                        capacity=None):
    """Add non-dominated points to filter; prune if over capacity."""
    capacity = capacity or len(xs)
    for x, y in zip(xs, ys):
        if not any(dominates(fy, y) for fy in filter_ys):
            filter_xs.append(x)
            filter_ys.append(y)
    # Keep only non-dominated in filter
    fxs2, fys2 = [], []
    for x, y in zip(filter_xs, filter_ys):
        if not any(dominates(fy, y) for fy in filter_ys
                    if not np.array_equal(fy, y)):
            fxs2.append(x); fys2.append(y)
```


15.4.4 Niche Techniques

Problem: Nicheing

Population methods may converge to a few *niches* (clusters) in the criterion space, failing to cover the full Pareto frontier.

Fitness Sharing

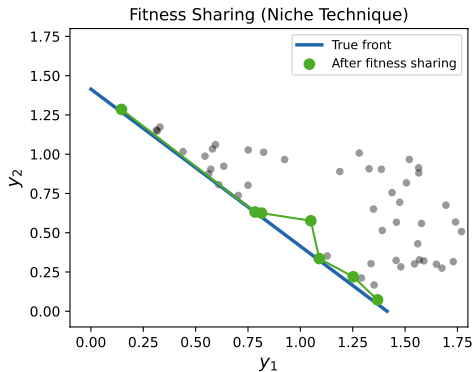
Penalize individuals by the number of nearby points in criterion space:

$$\tilde{f}_i(x) = f_i(x) \times (1 + \alpha \cdot |\mathcal{N}_\sigma(x)|)$$

where $\mathcal{N}_\sigma(x) = \{x' : \|f(x') - f(x)\| < \sigma\}$.

Equivalence Class Sharing

When two individuals tie on nondomination rank, prefer the one with fewer nearby points $\Rightarrow$ spreads population.



Fitness sharing significantly improves Pareto coverage.

15.5 Preference Elicitation: Overview

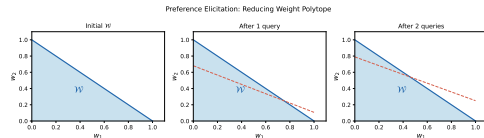
Goal

Infer a scalar objective from expert preferences about trade-offs, rather than specifying weights directly.

Model

Use weighted-sum model: $f(x) = w^\top f(x)$.

- Ask expert: “Do you prefer design a or b?”
 - Each preference is a constraint on w .
 - Accumulate constraints $\Rightarrow$ narrow w .
-
- Expert queries present pairs of Pareto-optimal points in criterion space.
 - Binary preference minimizes cognitive burden on expert.



Queries reduce the polytope of feasible weights $\mathcal{W}$.

15.5.1 Model Identification

Preference Constraints (Eq. 15.11–15.12)

Given pairs $\{(a^{(i)}, b^{(i)})\}_{i=1}^n$ where $a^{(i)}$ is preferred to $b^{(i)}$:

$$w^\top a^{(i)} < w^\top b^{(i)} \implies (a^{(i)} - b^{(i)})^\top w < 0$$

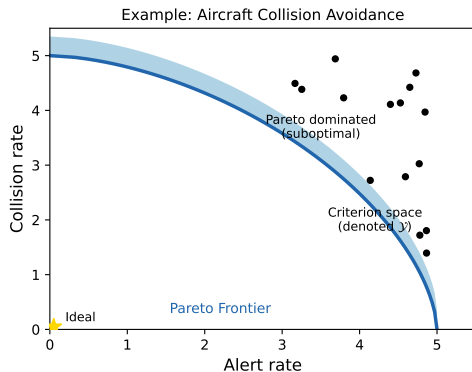
Consistent weight vector satisfies:

$$\begin{cases} (a^{(i)} - b^{(i)})^\top w < 0 & \forall i \\ 1^\top w = 1 \\ w \geq 0 \end{cases}$$

Optimal w (Eq. 15.13)

Choose w that **best separates** preferred from non-preferred:

$$\min \sum_{i=1}^n (a^{(i)} - b^{(i)})^\top w \quad \text{s.t. above constraints}$$



Each query adds a half-space constraint on w .

Preference Elicitation in Python

```
import numpy as np
from scipy.optimize import linprog

def update_weights(prev_w, a_pts, b_pts):
    """
    Find w minimizing ||w - prev_w||_1 subject to:
        (a_i - b_i)^T w < 0 for all i (a preferred over b)
        1^T w = 1, w >= 0
    Uses linprog (L1 objective via auxiliary variables).
    """
    n_pairs = len(a_pts)
    m = len(prev_w)
    # Inequality: (a_i - b_i)^T w <= -eps
    eps = 1e-6
    A_ub = np.array([a - b for a, b in zip(a_pts, b_pts)])
    b_ub = -eps * np.ones(n_pairs)
    # Equality: sum(w) = 1
    A_eq = np.ones((1, m))
    b_eq = np.array([1.0])
    # Minimize sum(w) as proxy (simple version)
    c = np.zeros(m)
    bounds = [(0, 1)] * m
    result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                     A_eq=A_eq, b_eq=b_eq,
                     bounds=bounds, method='highs')
    return result.x if result.success else prev_w
```

15.5.2 Paired Query Selection: Q-Eval

Q-Eval Algorithm

Goal: select query (a, b) that maximally reduces $|\mathcal{W}|$.

- 1 Compute **analytic center** c of $\mathcal{W}$:

$$c = \arg \max_{w \in \mathcal{W}} \sum_{i=1}^n \ln \left[(b^{(i)} - a^{(i)})^\top w \right]$$

- 2 For each candidate pair, compute normal distance from bisecting hyperplane to c .
- 3 Sort pairs by distance; evaluate k closest.
- 4 Choose pair with **volume ratio closest to 1** (most equal split).

Polyhedral Method

Approximates $\mathcal{W}$ with a bounding **ellipsoid** centered at analytic center. Choose queries perpendicular to longest ellipsoid axis $\Rightarrow$ reduces uncertainty fastest.

Stopping criteria

Minimax regret (Eq. 15.16):

$$x^* = \arg \min_x \max_{w \in \mathcal{W}} \max_{x' \in \mathcal{X}} w^\top (f(x) - f(x'))$$

Stop when minimax regret $< \theta$.

15.5.3 Design Selection

Decision Quality Improvement (Eq. 15.15)

Commit to x^* that minimizes worst-case objective:

$$x^* = \arg \min_{x \in \mathcal{X}} \max_{w \in \mathcal{W}} w^\top f(x)$$

Provides an **upper bound** on objective value.

Minimax Regret (Eq. 15.16)

Minimize the **maximum regret** of choosing x instead of x' :

$$\text{Regret}(x, x', w) = w^\top f(x) - w^\top f(x')$$

$$x^* = \arg \min_x \max_{w \in \mathcal{W}} \max_{x' \in \mathcal{X}} \text{Regret}(x, x', w)$$

Comparison

- DQI: *conservative* — protect against worst case.
- Minimax Regret: accounts for *uncertainty in utility*.
- Both are robust to incorrect weight estimates.

Key use

Minimax regret can also serve as a **stopping criterion**: terminate elicitation when $\text{regret} < \theta$.

Chapter 15 Summary

Core Concepts

- **Dominance:** x dominates x' if better on all, strictly better on at least one objective.
- **Pareto frontier:** set of non-dominated designs; boundary of criterion space $\mathcal{Y}$.
- **Utopia point:** component-wise optima (usually unattainable).

Scalarization Methods

- Constraint / ε -constraint: handles nonconvex frontiers.
- Lexicographic: priority ordering.
- Weighted sum: simple but misses nonconvex regions.
- Weighted min-max (Tchebycheff): complete coverage.

Population Methods

- VEGA: subpopulation specialization.
- Nondomination ranking (NSGA-II): multi-level selection.
- Pareto filter: elite archive.
- Fitness sharing: niche techniques for spread.

Preference Elicitation

- Binary queries constrain weight polytope $\mathcal{W}$.
- Q-Eval / Polyhedral: smart query selection.
- Minimax regret: robust design selection & stopping criterion.

Exercises: Key Problems

Exercise 15.1 — Weighted Sum Limitation

Q: What is a disadvantage of the weighted sum method for computing the Pareto frontier?

A: It cannot find Pareto-optimal points in *nonconvex* regions of the Pareto frontier. Linear contours cannot probe concave sections of $\mathcal{Y}$.

Exercise 15.2 — Why Population Methods?

Q: Why are population methods well-suited for multiobjective optimization?

A: Non-population methods find only a *single* Pareto point. Population methods naturally spread across the frontier, providing a representative sample of trade-offs in a single run.

Exercise 15.3 — Dominance Example

Points $\{[1, 2], [2, 1], [2, 2], [1, 1]\}$. Which are Pareto optimal?

A: Only $[1, 1]$ is strictly Pareto optimal. $[1, 2]$ and $[2, 1]$ are *weakly* Pareto optimal. $[2, 2]$ is dominated.

Exercise 15.7: Constraint Method Pareto Curve

Problem

$\min_x [x^2, (x-2)^2]$. Use the constraint method to obtain the Pareto curve.

Solution

Fix $f_2 \leq c$, minimize $f_1 = (x-2)^2$:

$$x^* = \begin{cases} 2 & c \geq 4 \\ \sqrt{c} & c \in [0, 4) \\ \text{infeasible} & c < 0 \end{cases}$$

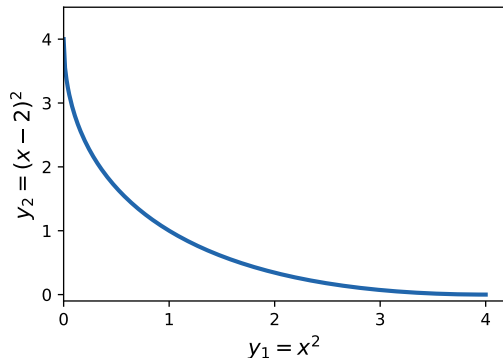
Pareto curve parametrized by $c \in [0, 4]$:

$$y_1 = c = (x^*)^2$$

$$y_2 = (\sqrt{c} - 2)^2 \quad \text{for } c < 4$$

Curve goes from $(0, 4)$ to $(4, 0)$

Ex. 15.7: Pareto Curve via Constraint Method



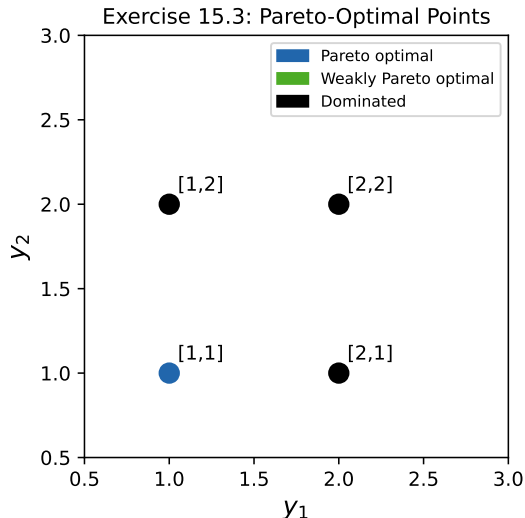
Pareto curve for $[x^2, (x-2)^2]$.

Exercise 15.3 Visualization

Example: 4 Criterion Points

Points: $\{[1, 2], [2, 1], [2, 2], [1, 1]\}$

- $[1, 1]$: Pareto optimal (dominates $[1, 2]$, $[2, 1]$, $[2, 2]$).
- $[1, 2]$: Weakly Pareto optimal (not dominated, but $[1, 1]$ dominates it).
- $[2, 1]$: Weakly Pareto optimal.
- $[2, 2]$: Dominated by $[1, 1]$.



Chapter 15: Key Takeaways

- 1 **Dominance** is the fundamental partial ordering for multiobjective problems.
- 2 The **Pareto frontier** — not a single point — is the solution to a multiobjective problem.
- 3 **Scalarization methods** (constraint, weighted sum, Tchebycheff) reduce the problem to single-objective optimization. Tchebycheff/min-max is the most general.
- 4 **Population methods** naturally spread across the Pareto frontier. NSGA-II (nondomination ranking + crowding distance) is the most widely used.
- 5 **Preference elicitation** infers a decision-maker's utility from pairwise comparisons, narrowing the weight polytope $\mathcal{W}$.
- 6 Design selection via **minimax regret** is robust to incomplete preference information.

Next Chapter

Chapter 16: Sampling Plans — how to efficiently explore the design space when function evaluations are expensive (space-filling designs, LHS, quasi-random sequences).